QuillAudits

AUDIT REPORT

May 2026

For

Floors

Table of Content

Executive Summary

Project Name	Floors Finance
Protocol Type	Bonding Curve, Staking
Project URL	https://www.floors.finance/
Overview	Floors Markets is a modular DeFi system built around a floor-backed issuance token. The core Floor contract uses a segmented discrete bonding curve and virtual collateral accounting to price buys/sells and expose a canonical floor price to other modules. The Credit Facility lets users lock issuance tokens and borrow collateral at configured LTV, including looped buy-and-borrow flows that create loan tranches. The Presale module manages early distribution across whitelist/public/closed states, combining direct buys, looped positions, dynamic fee multipliers, Merkle self-registration, and tranche claims unlocked by breakpoints or a time fallback. The Staking Manager integrates approved ERC4626 strategies so users can deploy floor-derived collateral, harvest yield, and maintain positions as prices evolve. Access control is handled by Authorizer modules (including token-gated variants), while treasuries route protocol fees. Governance and upgradeability rely on factory-driven deployment, beacon/proxy patterns, and emergency reverter endpoints. Together, the architecture separates pricing, credit, distribution, and yield into composable contracts that can be configured per market with common security and administration rails.
Audit Scope	The scope of this Audit was to analyze the Floors Finance Smart Contracts for quality, security, and correctness.
Source Code link	https://github.com/Floors-Finance/floors-sc/pull/125
Branch	quillaudits-audit
Commit Hash	43c0317
Language	Solidity

Contracts in Scope

src/proxies/interfaces/IInverterProxyAdmin_v1.sol
src/proxies/interfaces/
IInverterTransparentUpgradeableProxy_v1.sol
src/proxies/interfaces/IInverterBeacon_v1.sol
src/proxies/InverterBeaconProxy_v1.sol
src/proxies/InverterBeacon_v1.sol
src/external/reverter/InverterReverter_v1.sol
src/external/governance/Governor_v1.sol
src/external/forwarder/TransactionForwarder_v1.sol
src/factories/FloorFactory_v1.sol
src/factories/ModuleFactory_v1.sol
src/external/token/ERC20Issuance_v1.sol
src/factories/DeterministicFactory_v1.sol
src/external/governance/interfaces/IGovernor_v1.sol
src/external/forwarder/interfaces/
ITransactionForwarder_v1.sol
src/core/treasuries/abstracts/BaseTreasury_v1.sol
src/core/modules/interfaces/ICreditFacility_v1.sol
src/core/modules/interfaces/IPresale_v1.sol
src/core/issuance/BCDiscreteRedeemingVirtualSupplyv1.sol
src/factories/interfaces/IDeterministicFactory_v1.sol
src/core/authorizer/AUTRolesv2.sol
src/core/issuance/types/PackedSegment_v1.sol
src/core/treasuries/SplitterTreasury_v1.sol
src/core/treasuries/interfaces/ITreasury_v1.sol
src/core/treasuries/interfaces/ISplitterTreasury_v1.sol
src/core/modules/Presale_v1.sol
src/core/modules/CreditFacility_v1.sol
src/core/issuance/abstracts/
VirtualCollateralSupplyBase_v1.sol
src/factories/interfaces/IFloorFactory_v1.sol
src/core/issuance/abstracts/IssuanceBase_v2.sol
src/factories/interfaces/IModuleFactory_v1.sol
src/core/issuance/abstracts/
RedeemingIssuanceBase_v2.sol
src/external/token/interfaces/IERC20Issuance_v1.sol
src/core/issuance/libraries/PackedSegmentLib.sol
src/core/authorizer/AUTTokenGatedRoles_v2.sol
src/core/modules/lib/LibMetadata.sol
src/core/issuance/Floor_v1.sol
src/core/issuance/libraries/FixedPointMathLib.sol
src/core/modules/base/Module_v2.sol
src/core/modules/base/IModule_v2.sol
src/core/issuance/interfaces/IIssuanceBase_v2.sol
src/core/issuance/interfaces/
IRedeemingIssuanceBase_v2.sol
src/core/authorizer/interfaces/IAUTTokenGatedRoles_v2.sol

	src/core/issuance/interfaces/IDiscreteCurveMathLib_v1.sol src/core/issuance/interfaces/ IVirtualCollateralSupplyBase_v1.sol src/core/issuance/interfaces/ IBCDiscreteRedeemingVirtualSupplyv1.sol src/core/authorizer/interfaces/IAuthorizer_v2.sol src/core/issuance/formulas/DiscreteCurveMathLib_v1.sol src/core/issuance/interfaces/IFloor_v1.sol
Blockchain	Avalanche, HyperEVM, Ethereum
Method	Manual Analysis, Functional Testing, Automated Testing
Review 1	24th February - 13th April, 2026
Updated Code Received	24th April - 30th April, 2026
Review 2	24th April - 5th May, 2026
Fixed In	fix/178

Verify the Authenticity of Report on QuillAudits Leaderboard:

https://www.quillaudits.com/leaderboard

Number of Issues per Severity

Critical	0 (0.0%)
High	0 (0.0%)
Medium	10 (58.8%)
Low	0 (0.0%)
Informational	7 (41.2%)

Severity

Issues	Critical	High	Medium	Low	Informational
Open	0	0	0	0	0
Acknowledged	0	0	**5**	0	**1**
Partially Resolved	0	0	0	0	0
Resolved	0	0	**5**	0	**6**

Summary of Issues

Issue No.	Issue Title	Severity	Status
1	No Aave incentive rewards claiming	Medium	Resolved
2	Unbounded _userLoans Array Enables DoS on Full Repayment and Consolidation	Medium	Resolved
3	Aave Pool address hardcoded at init	Medium	Resolved
4	Empty strategies can be griefed with dust attack to block strategy removal	Medium	Resolved
5	addStrategy non-transferability probe is unreliable	Medium	Acknowledged
6	transferLoan allows newBorrower == CreditFacility	Medium	Acknowledged
7	Minters can burn arbitrary user balances without consent	Medium	Acknowledged
8	spendAllowance(): Any allowed minter can reduce arbitrary owner → spender allowance (griefing/control risk).	Medium	Acknowledged
9	IPool Aave ReserveData introduces ABI compatibility	Medium	Resolved
10	Blacklisted recipient can DoS SplitterTreasury distributions	Medium	Acknowledged

Summary of Issues

Issue No.	Issue Title	Severity	Status
11	Redundant issuance token forceApprove in _buyAndBorrow	Informational	Resolved
12	Use Ownable2Step instead of Ownable for safer ownership transfer	Informational	Resolved
13	Minor naming mismatch in timelock modifier argument	Informational	Resolved
14	Predictable CREATE2 floor proxy address	Informational	Acknowledged
15	Missing explicit zero-address checks in beacon constructor / upgrade path	Informational	Resolved
16	Governor init omits __ERC165_init	Informational	Resolved
17	Misnamed error in withdrawCollateralTo()	Informational	Resolved

Checked Vulnerabilities

- Access Management
- Arbitrary write to storage
- Centralization of control
- Ether theft
- Improper or missing events
- Logical issues and flaws
- Arithmetic Computations Correctness
- Race conditions/front running
- SWC Registry
- Re-entrancy
- Timestamp Dependence
- Gas Limit and Loops
- Exception Disorder
- Gasless Send
- Use of tx.origin
- Malicious libraries
- Compiler version not fixed
- Address hardcoded
- Divide before multiply
- Integer overflow/underflow
- ERC's conformance
- Dangerous strict equalities
- Tautology or contradiction
- Return values of low-level calls

- ✓ Missing Zero Address Validation
- ✓ Private modifier
- ✓ Revert/require functions
- ✓ Multiple Sends
- ✓ Using suicide
- ✓ Using delegatecall
- ✓ Upgradeable safety
- ✓ Using throw
- ✓ Using inline assembly
- ✓ Style guide violation
- ✓ Unsafe type inference
- ✓ Implicit visibility level

Techniques and Methods

Throughout the audit of smart contracts, care was taken to ensure:

- The overall quality of code
- Use of best practices
- Code documentation and comments, match logic and expected behavior
- Token distribution and calculations are as per the intended behavior mentioned in the whitepaper
- Implementation of ERC standards
- Efficient use of gas
- Code is safe from re-entrancy and other vulnerabilities

The following techniques, methods, and tools were used to review all the smart contracts:

Structural Analysis

In this step, we have analyzed the design patterns and structure of smart contracts. A thorough check was done to ensure the smart contract is structured in a way that will not result in future problems.

Static Analysis

A static Analysis of Smart Contracts was done to identify contract vulnerabilities. In this step, a series of automated tools are used to test the security of smart contracts.

Code Review / Manual Analysis

Manual Analysis or review of code was done to identify new vulnerabilities or verify the vulnerabilities found during the static analysis. Contracts were completely manually analyzed, their logic was checked and compared with the one described in the whitepaper. Besides, the results of the automated analysis were manually verified.

Gas Consumption

In this step, we have checked the behavior of smart contracts in production. Checks were done to know how much gas gets consumed and the possibilities of optimization of code to reduce gas consumption.

Tools and Platforms Used for Audit

Remix IDE, Foundry, Solhint, Mythril, Slither, Solidity Static Analysis.

Types of Severity

Every issue in this report has been assigned to a severity level. There are five levels of severity, and each of them has been explained below.

Critical: Immediate and Catastrophic Impact

Critical issues are the ones that an attacker could exploit with relative ease, potentially leading to an immediate and complete loss of user funds, a total takeover of the protocol's functionality, or other catastrophic failures. Critical vulnerabilities are non-negotiable; they absolutely must be fixed.

High (H): Significant Risk of Major Loss or Compromise

High-severity issues represent serious weaknesses that could result in significant financial losses for users, major malfunctions within the protocol, or substantial compromise of its intended operations. While exploiting these vulnerabilities might require specific conditions to be met or a moderate level of technical skill, the potential damage is considerable. These findings are critical and should be addressed and resolved thoroughly before the contract is put into the Mainnet.

Medium (M): Potential for Moderate Harm Under Specific Circumstances

Medium-severity bugs are loopholes in the protocol that could lead to moderate financial losses or partial disruptions of the protocol's intended behavior. However, exploiting these vulnerabilities typically requires more specific and less common conditions to occur, and the overall impact is generally lower compared to high or critical issues. While not as immediately threatening, it's still highly recommended to address these findings to enhance the contract's robustness and prevent potential problems down the line.

Low (L): Minor Imperfections with Limited Repercussions

Low-severity issues are essentially minor imperfections in the smart contract that have a limited impact on user funds or the core functionality of the protocol. Exploiting these would usually require very specific and unlikely scenarios and would yield minimal gain for an attacker. While these findings don't pose an immediate threat, addressing them when feasible can contribute to a more polished and well-maintained codebase.

Informational (I): Opportunities for Improvement, Not Immediate Risks

Informational findings aren't security vulnerabilities in the traditional sense. Instead, they highlight areas related to the clarity and efficiency of the code, gas optimization, the quality of documentation, or adherence to best development practices. These findings don't represent any immediate risk to the security or functionality of the contract but offer valuable insights for improving its overall quality and maintainability. Addressing these is optional but often beneficial for long-term health and clarity.

Types of Issues

Open

Security vulnerabilities identified that must be resolved and are currently unresolved.

Resolved

These are the issues identified in the initial audit and have been successfully fixed.

Acknowledged

Vulnerabilities which have been acknowledged but are yet to be resolved.

Partially Resolved

Considerable efforts have been invested to reduce the risk/impact of the security issue, but are not completely resolved.

Severity Matrix

Impact

Likelihood	High	Medium	Low
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium** - only a small amount of funds can be lost (such as leakage of value) or a core functionality of the protocol is affected.
- **Low** - can lead to any kind of unexpected behavior with some of the protocol's functionalities that's not so critical.

Likelihood

- High - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- Medium - only a conditionally incentivized attack vector, but still relatively likely.
- Low - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.

Medium Severity Issues

No Aave incentive rewards claiming

Resolved

Path

AaveStrategy_v1.sol || StakingManager_v1.sol

Function Name

`Missing claimRewards()`

Description

The strategy uses Aave's Pool to supply and withdraw the collateral token and tracks value via aToken balance. It never calls Aave's RewardsController to claim extra reward tokens (e.g. AAVE / partner tokens) that markets often pay on top of lending yield.

Impact

Those incentive rewards are never claimed through this contract and get stuck in the contract. The protocol and users miss that yield.

Likelihood

High on chains where Aave incentivizes the supplied asset. Not really an "exploit"—it's a missing feature.

Recommendation

Add a simple permissioned function that claims rewards to feeTreasury (or another fixed recipient), using the correct RewardsController address and asset list for deployment.

Unbounded _userLoans Array Enables DoS on Full Repayment and Consolidation

Resolved

Path

CreditFacility_v1.sol

Function Name

`borrowFor()`, `_createLoan()`, `_removeLoanFromUserLoans()`

Description

A permissioned caller can invoke borrowFor with any target address and a dust amount, spamming the victim's _userLoans array. Because _removeLoanFromUserLoans performs a linear scan over this array, both _processFullRepayment and _consolidateLoans will eventually revert out-of-gas, permanently locking the victim's collateral.

borrowFor allows any permissioned caller to assign loans to an arbitrary borrower_ with no minimum loan size beyond non-zero. _removeLoanFromUserLoans called by both _processFullRepayment and _consolidateLoans , performs a full linear scan over this array.

Note that _processPartialRepayment does not call _removeLoanFromUserLoans and is therefore unaffected.

Once the array grows past the block gas limit threshold, the victim cannot fully repay any loan or consolidate positions, locking their collateral with no in-protocol recovery path. Partial repayments remain functional but cannot close a loan, leaving collateral permanently locked.

Impact

Medium

Likelihood

Medium

Recommendation

- Minimum loan amount — raise the economic cost of spam to a prohibitive level.
- Cap _userLoans length — revert in _createLoan if the array exceeds MAX_LOANS_PER_USER so the user can close loans.

Aave's Pool address hardcoded at init

Resolved

Path

AaveStrategy_v1.sol

Function Name

`init(), _deployToStrategy() and _withdrawFromStrategy`

Description

The strategy saves _aavePool once in init from configData_ (only non-zero check). It does not read the current Pool from Aave's PoolAddressesProvider getPool(). Aave can point getPool() at a new Pool after governance; this contract keeps using the Pool address set at initialization forever unless upgraded.

Impact

After a Pool migration, deposits/withdrawals may fail or target a deprecated Pool while liquidity lives on the new one. Breaks yield path until governance upgrades or redeploys; not a normal user exploit.

Likelihood

Medium short-term, higher over years if Aave migrates the Pool on that network.

Recommendation

Prefer storing PoolAddressesProvider and resolving getPool() each call.

Floors Finance Team Feedback

https://github.com/Floors-Finance/floors-sc/issues/177

Auditor's Review

The inclusion of an emergencyMode + partial withdrawals + local balance tracking reduces the attack surface described in the issue trail (#134, #167) and improved handling of a misconfigured / exploited Aave pool.

Empty strategies can be griefed with dust attack to block strategy removal

Resolved

Path

StakingManager_v1.sol, AaveStrategy_v1.sol

Function Name

`removeStrategy(...)`

Description

removeStrategy() hard-reverts unless IStrategyBase_v1(strategy_).totalAssets() == 0. For AaveStrategy_v1, totalAssets() is _aToken.balanceOf(address(this)). Because aTokens are transferable ERC20s, any external account can transfer a tiny amount of aToken dust directly to the strategy address. This makes totalAssets() > 0 and permanently blocks removeStrategy() unless that dust is somehow removed.

Even legitimate rounding/dust from strategy math could leave non-zero residuals and trigger the same block condition, keeping removeStrategy() from executing.

Impact

Low for user funds: griefing ops and delays sunsetting

Likelihood

High, as it is cheap to execute by an attacker; Medium for residual dust

Recommendation

Do not gate removal only on raw totalAssets(). Instead, check for strategy's totalSupply() == 0 (no staker positions), and/or a permissioned skim on the strategy that withdraws or transfers stray aToken

addStrategy non-transferability probe is unreliable

Acknowledged

Path

StakingManager_v1.sol

Function Name

`addStrategy(...)`

Description

The try-catch block for transfer() may be insufficient to catch the expected heuristic i.e. strategy shares are non-transferrable. This is because the current check uses:

```
try IERC20(strategy_).transfer(address(0), 0) {
     revert StrategySharesTransferable();
 } catch {}
```

This treats any revert as proof of non-transferability but:
(1) vanilla ERC20 tokens revert on transfers to address(0).
(2) pausable, blacklisted tokens will fail the try block and continue execution added as strategies even if intended to not be transferable.
So the probe does not reliably enforce “strategy shares cannot be transferred.”

Impact

High, a transferable share token could be whitelisted. StakingManager accounting assumes balanceOf(user) matches that user’s stake and transfers will break that model.

Likelihood

Medium. addStrategy() is permissioned, so this is mainly misconfiguration / privileged-actor risk, but realistic in active development and integrations.

Recommendation

Best fix is an explicit capability check (e.g., strategy exposes sharesAreTransferable() == false) or strict strategy implementation allowlisting, not a simple probe on transfer(address(0), 0).

Floors Finance Team Feedback

In general the assumption here is that the strategies are trusted. The check here is essentially a curtesy. Your comments make sense.
I don’t know if the Explicit check for shares are transferable is doing anything, because you could just put a false in there and then the check would fit.
I think it’s probably best to just remove this, because we already test the interface compatibility of the ERC165.

transferLoan allows newBorrower == CreditFacility

Acknowledged

Path

CreditFacility_v1.sol

Function Name

`transferLoan()`

Description

transferLoan only rejects newBorrower_ == currentBorrower (msg.sender). It does not reject newBorrower_ == address(this) i.e. the CreditFacility contract.
If the borrower transfers the loan to the facility address, loan.borrower becomes the module. loanIdIsValid then requires msg.sender == loan.borrower, so only a caller whose msg.sender is the facility can use repay / rebalance / transferLoan on that loan. Normal EOAs cannot satisfy that. The contract has no path for users to keep operating the loan, so locked issuance and debt handling for that loan are effectively bricked by a mistaken or malicious recipient choice.

Impact

Self-inflicted / user error scenario: one bad transfer can make a loan unmanageable and lock associated accounting. Severity depends on whether to treat this as user mistake only; no profit for a third party unless they trick the borrower into passing the facility address

Likelihood

Low for accidental use; permissioned borrower only.

Recommendation

Revert if newBorrower_ == address(this).

Floors Finance Team Feedback

This is essentially “if somebody send their tokens to the wrong address they are gone”.
Will probably not tackle this.

Minters can burn arbitrary user balances without consent

Acknowledged

Path

ERC20Issuance_v1.sol

Function Name

`burn()`

Description

burn() is gated only by onlyMinter, and then calls _burn(_from, _amount) with no allowance/signature/ ownership checks. Any address in the allowedMinter mapping can burn tokens.

If a minter is compromised or misconfigured, an attacker can completely destroy user balances and break accounting expectations.

Impact

High; a privileged minter can destroy user balances, target specific users to grief them

Likelihood

Low

Recommendation

Privileges can be split to include a BURN_ROLE in addition to the MINT_ROLE or document clearly the trust assumption within the code.

Floors Finance Team Feedback

It makes sense to tag this, just because of the centralization risk of this Role, but I think it doesnt make that much of a difference to separate both of these access rights.
If the minter role is compromised, then the whole system can be completely exploited even if the allowance role is separate from that.
Will probably don't do anything about this. We try to mitigate the centralization risk by using proper access structures for distributing these roles (Multisig and Hierarchy Structures).

spendAllowance(): Any allowed minter can reduce arbitrary owner → spender allowance (griefing/ control risk).

Acknowledged

Path

ERC20Issuance_v1.sol

Function Name

`spendAllowance(...)`

Description

An account with the minter role can reduce any arbitrary owner → spender balance causing a direct griefing attack on users.

Impact

High, griefing legitimate users

Likelihood

Low

Floors Finance Team Feedback

It makes sense to tag this, just because of the centralization risk of this Role, but I think it doesnt make that much of a difference to separate both of these access rights.
If the minter role is compromised, then the whole system can be completely exploited even if the allowance role is separate from that.
Will probably don't do anything about this. We try to mitigate the centralization risk by using proper access structures for distributing these roles (Multisig and Hierarchy Structures).

IPool Aave ReserveData introduces ABI compatibility

Resolved

Path

IPool.sol

Function Name

`init()`

Description

The local ReserveData struct omits newer fields (e.g., liquidationGracePeriodUntil, virtualUnderlyingBalance) but strategy decodes getReserveData(asset) and reads aTokenAddress while the comments reference additional non-legacy fields. If the protocol intends to use the legacy data struct, it should be explicitly described as well as the tradeoff that comes with using an older / likely to be deprecated version.

Reference

AAVE Data Types

Impact

High, decoded data would be garbled leading to a loss of funds and user functionality.

Likelihood

Low, if ABI mismatch with an already deployed newer Pool version.

Recommendation

Use the exact official interface for the deployed Aave version (or a safer method to fetch aToken address

Blacklisted recipient can DoS SplitterTreasury distributions

Acknowledged

Path

SplitterTreasury_v1.sol

Function Name

`fetchFunds()`

Description

fetchFunds distributes tokens in a loop using safeTransfer to every recipient. If one recipient address is blacklisted by a token (e.g., USDT blacklist), that transfer reverts and the whole transaction reverts. As a result, no recipient gets paid until the bad recipient is removed or token state changes.

Impact

Medium operational DoS: fee distribution is blocked for all recipients for the affected token while the blacklisted recipient remains configured.

Likelihood

Medium. Depends on centralized token blacklist/admin actions and recipient set, but realistic for blacklist-enabled stablecoins.

Recommendation

Use pull-based claims per recipient, or skip/fail-soft per recipient and track unpaid balances, so one blocked recipient cannot revert the whole batch.

Floors Finance Team Feedback

1. The chance that something is blacklisted is not that high
2. If this happens then we can just change the target recipient
3. It is a safety mechanism that makes sure we notice this is happening in the first place.

Informational Issues

Redundant issuance token forceApprove in _buyAndBorrow

Resolved

Path

CreditFacility_v1.sol

Function Name

`_buyAndBorrow()`

Description

After buy(), issuance tokens already sit on the CreditFacility. The loop then calls _issuanceToken.forceApprove(address(this), tokensReceived) claiming _borrow needs a self-allowance for transferFrom(this, this, ...). But _borrow skips safeTransferFrom when issuanceTokenProvider_ == address(this), so no transferFrom runs and the approval is unused. Extra gas and a misleading comment for maintainers.

Recommendation

Remove the forceApprove to address(this) and update or delete the comment.

Use Ownable2Step instead of Ownable for safer ownership transfer

Resolved

Path

ERC20Issuance_v1.sol

Function Name

`Ownership model`

Description

The contract uses single-step Ownable ownership transfer. This increases operational risk: ownership can be transferred to a wrong address in one tx, and control is immediately lost.

Recommendation

Use Ownable2Step instead of Ownable.

Minor naming/docs mismatch in timelock modifier argument

Resolved

Path

ERC20Issuance_v1.sol

Function Name

`Ownership model`

Description

The modifier validates a timelock period but uses a generic parameter name amt_. This is minor readability/docs friction because the revert is specifically Governor__InvalidTimelockPeriod(uint amt_);

Recommendation

Rename amt_ to timelockPeriod_ (and keep NatSpec aligned) to make intent explicit and reduce confusion during reviews.

Predictable CREATE2 floor proxy address

Acknowledged

Path

ModuleFactory_v1.sol , FloorFactory_v1.sol

Function Name

`createModuleProxy(), createFloor()`

Description

Floor proxies use CREATE2 from ModuleFactory; salt uses msg.sender (e.g.FloorFactory), floor_ (address(0) for the floor proxy), and a nonce, so the next address is computable from public data. That is expected for deterministic deploys. Arbitrary malicious bytecode cannot occupy the same address as the real proxy because CREATE2 hashes init code; same-address squatting needs identical proxy + beacon bytecode and mainly risks deployment griefing (factory deploy reverts if code already exists), not a standard fund-drain path. New proxies hold no user funds until init and later use.

The theoretical dos:
- An attacker precomputes this small deterministic set of future addresses (Set A),
- then brute-forces their own CREATE2 deployments to find a collision (Set B).

Deploying to a colliding address permanently bricks that slot if deployed first since CREATE2 reverts on non-empty addresses and the nonce never increments on revert, causing createFloor() to revert indefinitely. But absolute collision cost remains ~2^80 operations, keeping this theoretical.

Floors Finance Team Feedback

With the Factories being deployed via a beacon structure we have a way to adapt to this in the unlikely case it happens.
Will close this with this.

Recommendation

Allow for a way to upgrade in the case of a permanent brick

Missing explicit zero-address checks in beacon constructor / upgrade path

Resolved

Path

InverterBeacon_v1.sol

Function Name

`constructor(), upgradeTo()`

Description

The constructor sets _reverterAddress from reverter without an explicit reverter != address(0) check. owner is passed to Ownable(owner); rely on OpenZeppelin behavior for zero owner. For implementation, _setImplementation uses validImplementation, which requires code.length > 0, so address(0) reverts there, but upgradeTo does not document that as the zero-address guard. Operational mistakes (wrong reverter in deploy) are easier without explicit validation and NatSpec clarity

Recommendation

add explicit validAddress-style checks in the constructor for reverter and owner (if not already guaranteed by the parent), and optionally assert newImplementation != address(0) before or alongside validImplementation for clearer revert reasons in upgradeTo / _setImplementation.

Governor init omits __ERC165_init

Resolved

Path

Governor_v1.sol

Function Name

`init()`

Description

init calls __AccessControl_init() but not __ERC165_init() from ERC165Upgradeable. In OpenZeppelin Contracts Upgradeable v5.x, __ERC165_init is empty, so supportsInterface and storage are effectively unchanged today.

If a future OZ release adds logic in __ERC165_init, proxies upgraded without ever having called it on the first init could miss that chain.

Recommendation

Call __ERC165_init() once in init so the initializer chain matches OpenZeppelin's upgradeable pattern and stays safe across dependency upgrades.

Misnamed error in withdrawCollateralTo()

Resolved

Path

BC_Discrete_Redeeming_VirtualSupply_v1.sol

Function Name

`withdrawCollateralTo()`

Description

The function reverts on zero amount using Module__IssuanceBase__ InvalidDepositAmount(), which is semantically mismatched for a withdrawal path.

Recommendation

Introduce a dedicated withdrawal-specific error (e.g., Module__IssuanceBase__ InvalidWithdrawAmount()) or rename the existing error name to a neutral term.

Functional Tests

PreSale_v1.sol

- ✓ init function can be called only once
- ✓ creditFacility, endTimestamp, baseCommissionsBps, initialMultiplier, decayDuration, globalIssuanceCap, _perAddressIssuanceCap and _state is set inside__presale_init
- ✓ Should revert if endTimestamp_ > 0 && endTimestamp_ <= block.timestamp
- ✓ Should revert if baseCommissionBps_.length == 0
- ✓ Should revert if priceBreakpoints_.length != baseCommissionBps_.length - 1
- ✓ Should revert if baseCommissionBps_(I) > MAX_COMMISSION_BPS
- ✓ Should revert if initialMultiplier_ < _BPS
- ✓ Should revert if initialMultiplier_ > _MAX_INITIAL_MULTIPLIER
- ✓ Should revert if decayDuration_ == 0
- ✓ getPositionState should revert if positionId_ == 0 || positionId_ > _posId
- ✓ buyPresale should revert if feePaid_ >= deposit_
- ✓ buyPresale should revert if tokensMinted_ < minAmountOut_
- ✓ In buyPresale treasury should receive feePaid_
- ✓ In buyPresale reserveToken should transfer from user to Presale_v1 contract and then approved and transferred to floor contract
- ✓ IssuanceToken should be minted to Presale_v1 contract
- ✓ _globalIssuanceCap and _perAddressIssuanceCap should be respected after mint of IssuanceToken in buyPresale
- ✓ In buyPresale _globalIssuance and _issuanceBy(buyer) should be updated
- ✓ buyPresaleWithLoops should revert if loopCount_ == 0 || loopCount_ >= _baseCommissionBps.length
- ✓ buyPresale and buyPresaleWithLoops should revert if amount_ is zero
- ✓ buyPresale and buyPresaleWithLoops should revert if state is different from Whitelist and _msgSender is not merkleWhitelisted
- ✓ buyPresale and buyPresaleWithLoops should revert when _endTimestamp > 0 && block.timestamp >= _endTimestamp

- After buyPresaleWithLoops user should have debt
- buyPresaleWithLoops should revert if tokensMinted_ < minAmountOut_
- buyPresaleWithLoops should make looped position for buyer
- claimAll can be successful only when _state == PresaleState.Closed || (_endTimestamp > 0 && block.timestamp >= _endTimestamp)
- tranche should be claimable when _endTimestamp > 0 && block.timestamp >= _endTimestamp
- tranche is not claimable when state != PresaleState.closed
- Should revert if positionId_ == 0 || positionId_ > _posId
- Should revert if _msgSender != caller
- Direct tokens should be claimable instantly after _state == PresaleState.Closed || (_endTimestamp > 0 && block.timestamp >= _endTimestamp)
- When the tranche is claimable the loan should be transfer to owner(buyer)
- When the tranche is claimable _claimedTranches should be updated to prevent double claim.
- In setPresaleState _startDecay should start if state_ == PresaleState.Public && _startTime == 0
- setCreditFacility should revert if _state != PresaleState.NotOpen
- setCaps should revert if caller is not permissioned granted
- setBaseCommissionBpsAndPriceBreakpoints should revert if _state != PresaleState.NotOpen
- setMerkleRoot should succeed if the caller has permission and the _state == PresaleState.NotOpen

CreditFacility_v1.sol

- init can be called only once (initializer)
- init sets _collateralToken and _issuanceToken from floor, decodes configData_ to loanToValueRatio, maxLoops, borrowingFeeRate, and sets _nextLoanId = 1
- init should revert if loanToValueRatio_ > _MAX_BORROWABLE_QUOTA (10000 bps)
- init should revert if borrowingFeeRate_ > _MAX_FEE_BPS (5000)
- borrow / borrowFor should revert if requestedLoanAmount_ is zero (validAmount)

- ✓ borrowFor should revert if receiver_ is zero (validAddress)
- ✓ buyAndBorrow / buyAndBorrowFor should revert if amount_ is zero (validAmount)
- ✓ buyAndBorrow / buyAndBorrowFor should revert if loops_ < 1 || loops_ > _maxLoops (InvalidLoopCount)
- ✓ buyAndBorrowFor should revert if receiver_ is zero address (validAddress)
- ✓ loanIdIsValid should revert if loan inactive or borrower != _msgSender (InvalidLoanId)
- ✓ _borrow should revert if borrowingFee rounds to zero while _borrowingFeeRate > 0 (BorrowAmountTooSmall)
- ✓ On borrow, issuance tokens should be pulled from provider and loan created; collateral net of fee sent to receiver; fee to treasury via fetchFunds
- ✓ buyAndBorrow should pull collateral from _msgSender, loop Floor.buy then _borrow with tokenReceiver address(this); revert if remainingCollateral == 0 mid-loop (InsufficientCollateralForLoops)
- ✓ buyAndBorrow should revert if no issuance received from a buy (NoIssuanceTokensReceived)
- ✓ buyAndBorrow should revert if totalTokensMinted < minAmountOut_ (SlippageExceeded)
- ✓ buyAndBorrow should return leftover collateral to _msgSender after loops
- ✓ buyAndBorrow with consolidate_ true and multiple loops should return a single consolidated loan id
- ✓ repay should revert on invalid loan id or zero repaymentAmount_; pull collateral from user, depositCollateralTo floor, transfer unlocked issuance to user
- ✓ repay with repaymentAmount_ >= remaining should fully close loan, clear state, remove from _userLoans
- ✓ transferLoan should revert if newBorrower_ == currentBorrower (InvalidTransferRecipient)
- ✓ transferLoan should revert if newBorrower_ is zero (validAddress)
- ✓ rebalanceLoan should return 0 if loan inactive; else revert NothingToRebalance if remainingAmount >= releasableCollateralAmount
- ✓ rebalanceLoan on success should increase remainingLoanAmount to releasableCollateralAmount and send released collateral net of borrowing fee
- ✓ consolidateLoans should revert if loanIds_.length < 2 (InsufficientLoansToConsolidate)
- ✓ consolidateLoans should revert if any loan inactive or borrower mismatch (InvalidLoansForConsolidation)

- consolidateLoans should deactivate old loans, create one new loan with summed debt and locked issuance
- setLoanToValueRatio / setMaxLoops / setBorrowingFeeRate should be permissioned and use same validation as init setters
- Lower LTV via setLoanToValueRatio should emit LoanToValueRatioLowered when new ratio < old

CreditFacility_v1.sol

- init can be called only once (initializer)
- init sets _collateralToken and _issuanceToken from floor, decodes configData_ to loanToValueRatio, maxLoops, borrowingFeeRate, and sets _nextLoanId = 1
- init should revert if loanToValueRatio_ > _MAX_BORROWABLE_QUOTA (10000 bps)
- init should revert if maxLoops_ < 1 || maxLoops_ > type(uint8).max
- init should revert if borrowingFeeRate_ > _MAX_FEE_BPS (5000)
- borrow / borrowFor should revert if requestedLoanAmount_ is zero (validAmount)
- borrowFor should revert if receiver_ is zero (validAddress)
- buyAndBorrow / buyAndBorrowFor should revert if amount_ is zero (validAmount)
- buyAndBorrow / buyAndBorrowFor should revert if loops_ < 1 || loops_ > _maxLoops (InvalidLoopCount)
- buyAndBorrowFor should revert if receiver_ is zero address (validAddress)
- loanIdIsValid should revert if loan inactive or borrower != _msgSender (InvalidLoanId)
- _borrow should revert if borrowingFee rounds to zero while _borrowingFeeRate > 0 (BorrowAmountTooSmall)
- On borrow, issuance tokens should be pulled from provider and loan created; collateral net of fee sent to receiver; fee to treasury via fetchFunds
- buyAndBorrow should pull collateral from _msgSender, loop Floor.buy then _borrow with tokenReceiver address(this); revert if remainingCollateral == 0 mid-loop (InsufficientCollateralForLoops)
- buyAndBorrow should revert if no issuance received from a buy (NoIssuanceTokensReceived)
- buyAndBorrow should revert if totalTokensMinted < minAmountOut_ (SlippageExceeded)
- buyAndBorrow should return leftover collateral to _msgSender after loops

- ✓ buyAndBorrow with consolidate_ true and multiple loops should return a single consolidated loan id
- ✓ repay should revert on invalid loan id or zero repaymentAmount_; pull collateral from user, depositCollateralTo floor, transfer unlocked issuance to user
- ✓ repay with repaymentAmount_ >= remaining should fully close loan, clear state, remove from _userLoans
- ✓ transferLoan should revert if newBorrower_ == currentBorrower (InvalidTransferRecipient)
- ✓ transferLoan should revert if newBorrower_ is zero (validAddress)
- ✓ rebalanceLoan should return 0 if loan inactive; else revert NothingToRebalance if remainingAmount >= releasableCollateralAmount
- ✓ rebalanceLoan on success should increase remainingLoanAmount to releasableCollateralAmount and send released collateral net of borrowing fee
- ✓ consolidateLoans should revert if loanIds_.length < 2 (InsufficientLoansToConsolidate)
- ✓ consolidateLoans should revert if any loan inactive or borrower mismatch (InvalidLoansForConsolidation)
- ✓ consolidateLoans should deactivate old loans, create one new loan with summed debt and locked issuance
- ✓ setLoanToValueRatio / setMaxLoops / setBorrowingFeeRate should be permissioned and use same validation as init setters
- ✓ Lower LTV via setLoanToValueRatio should emit LoanToValueRatioLowered when new ratio < old
- ✗ In transferLoan newBorrower can be address(this)—>(CreditFacility_v1); loan then unusable by normal EOAs

StakingManager_v1 / StrategyBase_v1 / AaveStrategy_v1

- ✓ StakingManager init can only run once
- ✓ After init, collateral token matches floor collateral
- ✓ After init, issuance token matches floor issuance
- ✓ Init fails if performance fee is over 100%
- ✓ Init stores performance fee in bps correctly when valid
- ✓ Strategy uses same collateral token as the floor
- ✓ Deposit with zero amount should fail

- ✓ Deposit with address zero receiver should fail
- ✓ Deposit moves collateral from caller into the strategy
- ✓ Deposit sends pulled funds into the yield protocol (e.g. Aave supply)
- ✓ Withdraw with zero amount should fail
- ✓ Withdraw with address zero receiver should fail
- ✓ Withdraw pulls assets from the yield protocol first
- ✓ After withdraw, strategy shares are burned from the owner
- ✓ User receives floor tokens equal to what withdrawal collateral actually got
- ✓ Aave strategy init fails if pool address is zero
- ✓ Aave strategy init fails if that asset has no aToken on the pool
- ✓ After Aave init, getAavePool returns the pool you passed in
- ✓ After Aave init, getAToken returns the aToken for the collateral
- ✓ Supplying through the strategy increases aToken balance
- ✓ Withdrawing through the strategy decreases aToken balance
- ✓ totalAssets on Aave strategy equals aToken balance of the contract
- ✓ Cannot stake into a strategy that is not on the approved list
- ✓ Cannot stake zero issuance tokens
- ✓ Withdraw with zero amount should fail
- ✓ Stake pulls issuance tokens from the user
- ✓ Stake pulls the right amount of collateral out of the floor
- ✓ Stake deposits that collateral into the strategy for the user
- ✓ Stake increases locked issuance on the user position
- ✓ Stake increases deployed collateral on the user position
- ✓ Stake saves the floor price used for that step
- ✓ If user already has a position, a new stake runs rebalance first
- ✓ Rebalance only adds collateral when floor price went up vs deployed amount
- ✓ Rebalance sends extra collateral from floor into the strategy
- ✓ Rebalance updates last floor price on the position

- ✓ Harvest fails if user has no position in that strategy
- ✓ Harvest fails if strategy is not approved
- ✓ Harvest fails if there is nothing above principal to take as yield
- ✓ Harvest pulls yield from the strategy to the receiver_
- ✓ Harvest sends a cut to the treasury when fee is non-zero
- ✓ Withdraw funds fails if user has no position
- ✓ Withdraw funds fails if amount is zero
- ✓ Withdraw funds fails if asking for more than position value
- ✓ Withdraw funds runs harvest first when there is yield
- ✓ Withdraw funds returns collateral from strategy to manager then to floor
- ✓ Withdraw funds returns issuance tokens to the receiver
- ✓ Withdraw funds lowers locked issuance and deployed collateral
- ✓ Rebalance entrypoint fails if user has no position
- ✓ Rebalance entrypoint fails if floor did not move up enough to deploy more
- ✓ Rebalance entrypoint returns how much extra collateral was deployed
- ✓ addStrategy fails for zero address
- ✓ addStrategy fails if strategy was already added
- ✓ addStrategy fails if strategy asset is not the floor collateral
- ✓ removeStrategy fails if strategy was never added
- ✓ removeStrategy fails while strategy still holds any assets
- ✓ setPerformanceFeeBps fails if new fee is over 100%
- ✓ getPosition returns stored locked issuance and deployed collateral
- ✓ getPositionValue uses convertToAssets on user share balance
- ✓ getAvailableYield is current value minus deployed principal when positive
- ✓ isStrategyApproved is true only for strategies the admin added
- ✓ AaveStrategy has a function to claim Aave bonus reward tokens
- ✓ Pool address is dynamically generated
- ✓ removeStrategy can’t be blocked with 1-wei deposits
- ✓ getReserveData struct in our code matches real Aave pool on chain

Governor_v1.sol

- Governor init can only run once
- Init fails if community multisig address is zero
- Init fails if team multisig address is zero
- Init fails if timelock period is under 48 hours
- After init, community multisig has COMMUNITY_MULTISIG_ROLE
- After init, team multisig has TEAM_MULTISIG_ROLE
- Community role is admin of itself
- Community role is admin of DEFAULT_ADMIN_ROLE
- Community role is admin of TEAM_MULTISIG_ROLE
- After init, timelock period matches what was passed in
- After init, module factory address is saved
- Init fails if module factory address is zero
- Only the linked module factory can call moduleFactoryInitCallback
- moduleFactoryInitCallback fails if linked beacons were already set
- moduleFactoryInitCallback stores the full beacon list when all checks pass
- getModuleFactory returns the saved factory address
- getLinkedBeacons returns the array from the factory callback
- getBeaconTimelock returns timelock struct for a given beacon address
- setModuleFactory can only be called by community multisig
- setModuleFactory fails if new factory address is zero
- registerMetadataInModuleFactory can be called by community or team multisig
- registerMetadataInModuleFactory adds the beacon to linked beacons first
- getBeaconTimelock returns timelock struct for a given beacon address
- registerNonModuleBeacon can be called by community or team multisig
- registerNonModuleBeacon adds the beacon to linked beacons
- Register paths fail if beacon is not a contract
- Register paths fail if beacon owner is not this Governor
- upgradeBeaconWithTimelock can be called by community or team multisig

- ✓ upgradeBeaconWithTimelock fails if beacon address is not accessible
- ✓ upgradeBeaconWithTimelock fails if new implementation is address zero
- ✓ upgradeBeaconWithTimelock sets timelock active and end time now plus period
- ✓ upgradeBeaconWithTimelock saves intended implementation and version numbers
- ✓ Starting a new timelock while one is active emits overwrite event then replaces
- ✓ triggerUpgradeBeaconWithTimelock fails if no timelock was started
- ✓ triggerUpgradeBeaconWithTimelock fails if timelock end time is not reached yet
- ✓ triggerUpgradeBeaconWithTimelock clears timelock active and calls beacon upgradeTo
- ✓ triggerUpgradeBeaconWithTimelock passes override shutdown flag false
- ✓ triggerUpgradeBeaconWithTimelock can be called by community or team multisig
- ✓ cancelUpgrade can be called by community or team while timelock is active
- ✓ cancelUpgrade turns timelock active off for that beacon
- ✓ setTimelockPeriod can only be called by community multisig
- ✓ setTimelockPeriod fails if new value is under 48 hours
- ✓ initiateBeaconShutdown can be called by community or team for one beacon
- ✓ initiateBeaconShutdown fails if beacon is not accessible
- ✓ forceUpgradeBeaconAndRestartImplementation is only for community multisig
- ✓ Force upgrade fails if beacon not accessible or implementation is zero
- ✓ restartBeaconImplementation is only for community multisig
- ✓ restartBeaconImplementation fails if beacon not accessible
- ✓ acceptOwnership can be called by community or team multisig
- ✓ acceptOwnership fails if target is not a contract
- ✓ Random caller cannot use onlyLinkedModuleFactory functions
- ✓ Random caller cannot start timelock upgrade without role
- ✓ Random caller cannot trigger upgrade after timelock
- ✓ Random caller cannot shut down beacons without role

Floor_v1.sol (+ BC_Discrete_Redeeming_VirtualSupply_v1 + libs)

- Init runs once; parent decodes issuance token, collateral token, segments, buy fee, sell fee from configData_
- After init, buy and sell fee state matches what was passed in
- getSegments returns the live packed segment array
- getFloorSection is always _segments[0]
- getPremiumSections copies every segment after the first into a new array
- getFloorPrice is only the first segment start price (not the higher “buy” spot)
- getFloorMetrics returns floor price, floor supply cap, live issuance totalSupply, and next premium start price if it exists
- getStaticPriceForBuying looks up price at supply + 1 (next mint pays this)
- getStaticPriceForSelling looks up price at current supply (redeem uses this)
- buy pulls collateral from user, mints issuance, bumps virtual collateral by real token balance increase
- buyFor sends minted issuance to receiver_; zero receiver fails validReceiver
- Buy and sell revert when output would be below minAmountOut (slippage)
- sell burns user issuance and pays collateral to msg.sender; virtual collateral drops by real balance change
- Floor sellTo pays receiver_; virtual collateral drops by real balance change
- If total issuance drops below floor segment supply, floor segment supply is shrunk to match supply
- reconfigureSegments is permissioned; can pull new collateral from caller or mark it self-supplied
- Self-supplied reconfigure checks token balance covers virtual supply
- reconfigureSegments reverts if new curve needs more reserve than current virtual collateral allows
- withdrawCollateralTo sends collateral out to caller
- withdrawCollateralTo reverts on zero amount
- depositCollateralFrom pulls collateral in
- Parent _setSegments runs DiscreteCurveMathLib segment array validation then stores _segments
- Floor _setSegments adds rule: need at least 2 segments

- ✓ Floor _setSegments adds rule: first segment must have 1 step and 0 price step
- ✓ raiseFloor needs permission and non-zero collateral
- ✓ raiseFloor fails when fewer than 2 segments
- ✓ raiseFloor fails when collateral is below minimum for at least 1 wei price move
- ✓ raiseFloor computes exact reserve for new curve vs current supply and reverts if no real increase
- ✓ raiseFloor only pulls the exact extra collateral needed
- ✓ When floor supply would pass real issuance, supply is capped and only price can keep moving
- ✓ getStaticPriceForBuying looks up price at supply + 1 (next mint pays this)
- ✓ Absorption reverts if budget remains but there is no premium segment left to absorb
- ✗ In ERC20Issuance_v1.sol ·privileged users (MINTER_ROLE) can burn any arbitrary user's tokens
- ✗ In SplitterTreasury_v1.sol one blacklisted recipient reverts entire distribution

Automated Tests

No major issues were found. Some false positive errors were reported by the tools. All the other issues have been categorized above according to their level of severity.

Threat Model

1. External Dependencies & Trust Boundaries

This section enumerates everything the protocol does not fully control.

1.1 External Dependencies Table

ERC20Aument

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
IERC20 (OpenZeppelin)	Interface	Defines the ERC20 interface for collateral and issuance token interactions	Correct interface definition matching ERC20 standard	N/A
SafeERC20 (OpenZeppelin)	Library	Wraps ERC20 transfer calls with revert-on-failure checks (safeTransfer, safeTransferFrom, forceApprove)	Library correctly handles return values and reverts on failure	If library has a bug, token transfers may silently fail
Math (OpenZeppelin)	Library	Used for mulDiv precision-safe division in borrowing power and issuance token calculations	Correct rounding and overflow protection	Incorrect rounding could cause over/under-collateralization

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
Module_v2	Base Contract	Provides permissioned modifier, validAmount, validAddress, _msgSender (ERC2771), fee treasury access	Authorizer correctly enforces role-based access control	Misconfigured authorizer could allow unauthorized access
IFloor_v1 (Floor contract)	External Contract	Source of floor price via getFloorPrice(); collateral withdrawn via withdrawCollateralTo(); collateral deposited via depositCollateralFrom()	Floor price is accurate and not manipulable within a single tx; Floor has sufficient collateral reserves	Stale or manipulated floor price leads to incorrect loan sizing; insufficient reserves cause failed borrows
IIssuanceBase_v2 (Floor contract)	External Contract	Used to buy issuance tokens in buyAndBorrow loop; provides getCollateralToken() and getIssuanceToken()	buy() returns correct token amounts; bonding curve math is sound	Bonding curve manipulation could affect loop economics
ITreasury_v1	External Contract	Receives borrowing fees via fetchFunds()	Treasury correctly receives and holds fees	Failed fee transfer could block borrowing
Collateral Token	Collateral Token	Borrowed by users; transferred to/from Floor and Treasury	Standard ERC20 with no fee-on-transfer, no rebasing	Non-standard token behavior breaks loan accounting

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
Issuance Token	External ERC20	Locked as collateral by borrowers; returned on repayment	Standard ERC20 with no fee-on-transfer, no rebasing	Non-standard token behavior breaks loan accounting

Presale_v1

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
IERC20 (OpenZeppelin)	Interface	Defines the ERC20 interface for collateral and issuance token interactions	Correct interface definition matching ERC20 standard	N/A
SafeERC20 (OpenZeppelin)	Library	Wraps ERC20 transfer calls with revert-on-failure checks	Library correctly handles return values and reverts on failure	N/A
ReentrancyGuardUpgradeable (OpenZeppelin)	Base Contract	Prevents reentrancy on buyPresale, buyPresaleWithLoops, claimAll	Mutex logic is correctly implemented	N/A

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
Module_v2	Base Contract	Provides permissioned modifier, validAmount, _msgSender (ERC2771), fee treasury access	Authorizer correctly enforces role-based access control	Misconfigured authorizer could allow unauthorized access
DecayingFeeMultiplierBase_v1	Base Contract	Provides time-decaying fee multiplier for commission calculation	Multiplier decays correctly from initial value to 1x over configured duration	Incorrect decay math could over/under-charge fees
MerkleWhitelistBase_v1	Base Contract	Provides Merkle-tree-based whitelist verification for whitelist phase	Merkle root is correctly set; proof verification is sound	Incorrect Merkle root could exclude/include wrong addresses
ICreditFacility_v1	External Contract	Used for looped buy-and-borrow operations; loans are transferred on claim	CreditFacility correctly executes buyAndBorrow, getLoan, transferLoan; loan IDs remain valid	N/A
IFloor_v1 (Floor contract)	External Contract	Source of floor price for tranche unlock checks; buy() for direct purchases	Floor price is accurate; buy() returns correct amounts	Manipulated floor price could prematurely/never unlock tranches

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
IIssuanceBase_v2 (Floor contract)	External Contract	Used to buy issuance tokens (direct buy and final remainder buy)	buy() returns correct token amounts	Bonding curve manipulation could affect purchase economics
Collateral Token (Reserve)	External ERC20	Deposited by users for presale purchases	Standard ERC20, no fee-on-transfer, no rebasing	Non-standard token behavior breaks accounting
Issuance Token	External ERC20	Minted on purchase, transferred on direct claim	Standard ERC20, no fee-on-transfer, no rebasing	Non-standard token behavior breaks claim amounts

StakingManager_v1

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
IERC20 (OpenZeppelin)	Interface	Defines the ERC20 interface for collateral, issuance, and strategy share interactions	Correct interface definition matching ERC20 standard	N/A
SafeERC20 (OpenZeppelin)	Library	Wraps ERC20 transfer calls with revert-on-failure checks	Library correctly handles return values and reverts on failure	N/A

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
IERC4626 (OpenZeppelin)	Interface	Strategy vaults implement ERC4626 for deposit/ withdraw and convertToAssets	Strategy correctly implements ERC4626 share accounting	N/A
IERC165 (OpenZeppelin)	Interface	Used to verify strategy implements IStrategyBase_v1 interface	supportsInterface returns truthful results	N/A
ReentrancyGuardUpgradeable (OpenZeppelin)	Base Contract	Prevents reentrancy on stake, harvestYield, withdrawFunds, rebalance	Mutex logic is correctly implemented	N/A
Module_v2	Base Contract	Provides permissioned modifier, validAmount, validAddress, _msgSender (ERC2771), fee treasury	Authorizer correctly enforces role-based access control	Misconfigured authorizer could allow unauthorized access
IFloor_v1 (Floor contract)	External Contract	Source of floor price for collateral calculations in stake and rebalance	Floor price is accurate and not manipulable within a single tx	Manipulated floor price inflates/ deflates collateral deployment

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
IBC_Discrete_Redeeming_VirtualSupply_v1 (Floor contract)	External Contract	Withdraws collateral via withdrawCollateralTo() on stake/rebalance; deposits via depositCollateralFrom() on withdraw	Floor has sufficient collateral reserves; operations succeed atomically	Insufficient reserves block staking; failed deposit blocks withdrawal
IStrategyBase_v1	External Contract	Yield-generating strategies that hold deployed collateral; withdraw() used for yield harvesting and fund withdrawal	Strategies are non-malicious; withdraw() returns requested amounts; totalAssets() is accurate	N/A
Collateral Token	External ERC20	Withdrawn from Floor, deployed to strategies, harvested as yield	Standard ERC20, no fee-on-transfer, no rebasing	Non-standard token behavior breaks position accounting
Issuance Token	External ERC20	Staked by users, locked in contract, returned on withdrawal	Standard ERC20, no fee-on-transfer, no rebasing	Non-standard token behavior breaks position accounting

Floor_v1 / BC_Discrete_Redeeming_VirtualSupply_v1

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
IERC20 (OpenZeppelin)	Interface	Defines the ERC20 interface for collateral and issuance token interactions	Correct interface definition matching ERC20 standard	N/A
SafeERC20 (OpenZeppelin)	Library	Wraps ERC20 transfer calls with revert-on-failure checks (safeTransfer, safeTransferFrom, forceApprove)	Library correctly handles return values and reverts on failure	N/A
ReentrancyGuardUpgradeable (OpenZeppelin)	Base Contract	Prevents reentrancy on buyFor, buy, sellTo, sell via nonReentrant	Mutex logic is correctly implemented	N/A
Module_v2	Base Contract	Provides permissioned modifier, validAmount, _msgSender (ERC2771), fee treasury access	Authorizer correctly enforces role-based access control	Misconfigured authorizer could allow unauthorized access to buy/sell/admin functions
VirtualCollateralSupplyBase_v1	Base Contract (abstract)	Tracks _virtualCollateralSupply — the bonding curve's view of how much collateral backs issued tokens. Add/sub/set operations.	_virtualCollateralSupply accurately reflects the collateral the curve should consider	withdrawCollateralTo and depositCollateralFrom intentionally do NOT update virtual supply. Real balance can diverge from virtual supply.

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
IssuanceBase_v2	Base Contract (abstract)	Provides buy flow (_buyOrder), fee handling, _mint/_burn/_spendAllowance, buy fee, and token setters	Buy order correctly calculates purchase return and mints tokens	Fee misconfiguration (up to 100%) could consume entire deposit
RedeemingIssuanceBase_v2	Base Contract (abstract)	Provides sell flow (_sellOrder), sell fee handling	Sell order correctly calculates sale return and burns tokens	Fee misconfiguration could consume entire sell proceeds
DiscreteCurveMathLib_v1	Library	Core bonding curve math: _calculatePurchaseReturn, _calculateSaleReturn, _calculateReserveForSupply, _findPositionForSupply, _validateSegmentArray	Math is correct; no overflow in step calculations; rounding is protocol-favoring	Rounding dust accumulates over many transactions (protocol-favoring, intentional)
PackedSegmentLib	Library	Packs/unpacks segment data (initialPrice, priceIncrease, supplyPerStep, numberOfSteps) into PackedSegment type	Packing/unpacking is lossless and correct	N/A

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
FixedPointMathLib	Library	Used in _segmentAbsorption for _mulDivUp (ceiling division for collateral cost)	Correct ceiling division behavior	N/A
ERC20Issuance_v1	External ERC20	The issuance token the bonding curve mints and burns; must register this contract as an allowed minter	Contract is registered as allowed minter; standard ERC20 with mint/burn	If not registered as minter, all buy/sell operations fail; non-standard behavior breaks accounting
Collateral Token	External ERC20	Deposited by buyers, returned to sellers, held as reserve backing the bonding curve	Standard ERC20 with no fee-on-transfer, no rebasing	Non-standard token behavior breaks _virtualCollateralSupply tracking via balance-diff; fee-on-transfer causes reserve accounting mismatch
CreditFacility_v1 (caller)	External Module	Calls withdrawCollateralTo (borrow) and depositCollateralFrom (repay); calls buy in buyAndBorrow loops	Returns collateral via depositCollateralFrom after loan repayment	Collateral withdrawn but not yet returned creates window where virtual supply > real balance. If reconfigureSegments is called during this window, invariant check may accept invalid segments.

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
StakingManager_v1 (caller)	External Module	Calls withdrawCollateralTo (stake/rebalance) and depositCollateralFrom (withdrawFunds)	Returns collateral via depositCollateralFrom after unstaking	N/A

2. Entry & Exit Points Analysis (Function-Level)

This is the core of the threat model. Every externally callable function must appear here.

2.1 Function Entry / Exit Table

Each function gets one table.

CreditFacility_v1

Contract Name	Function	Category
CreditFacility_v1	init(address floor_, address authorizer_, address feeTreasury_, bytes memory configData_)	**What this function can do** Initialize the module with LTV ratio, max loops, borrowing fee rate; Fetch collateral/issuance tokens from Floor; set nextLoanId to **What this function cannot/should not do** Be called more than once (initializer modifier) accept LTV > 10000 bps, fee > 5000 bps, or loops outside 1–255. **Main invariant(s)** Module is initialized exactly once; all configuration parameters are within valid bounds. **Access level** External, initializer (once at deployment via proxy)
CreditFacility_v1	getLoanToValueRatio()	**What this function can do** Return the current LTV ratio in bps. **What this function cannot/should not do** Modify state. **Main invariant(s)** Returns the stored _loanToValueRatio. **Access level** Public view, unrestricted

Contract Name	Function	Category
CreditFacility_v1	getMaxLoops()	**What this function can do** Return the current max loops value. **What this function cannot/should not do** Modify state. **Main invariant(s)** Returns the stored _maxLoops. **Access level** Public view, unrestricted
CreditFacility_v1	getLoan(uint loanId_)	**What this function can do** Return loan details for a given loan ID. **What this function cannot/should not do** Modify state; revert on non-existent IDs (returns default struct). **Main invariant(s)** Returns accurate loan data. **Access level** Public view, unrestricted
CreditFacility_v1	getUserLoanIds(address user_)	**What this function can do** Return array of loan IDs for a user. **What this function cannot/should not do** Modify state. **Main invariant(s)** Returns all active loan IDs for the user. **Access level** Public view, unrestricted

Contract Name	Function	Category
CreditFacility_v1	Returns all active loan IDs for the user.	**What this function can do** Return array of full Loan structs for a user. **What this function cannot/should not do** Modify state. **Main invariant(s)** Each returned Loan matches the stored loan for that ID. **Access level** Public view, unrestricted
CreditFacility_v1	getUserTotalDebt(address user_)	**What this function can do** Return aggregate debt, locked tokens, and loan count for a user. **What this function cannot/should not do** Modify state. **Main invariant(s)** Sum of remainingLoanAmount and lockedIssuanceTokens across all user loans. **Access level** Public view, unrestricted

Contract Name	Function	Category
CreditFacility_v1	borrow(uint requestedLoanAmount_)	**What this function can do** Create a new loan; lock caller's issuance tokens proportional to requested amount / (LTV * floorPrice); withdraw collateral from Floor; charge borrowing fee to treasury; transfer net collateral to caller. **What this function cannot/should not do** zero amount; allow reentrancy; bypass access control; create a fee-less borrow when fee rate > 0 and amount is too small (reverts). **Main invariant(s)** lockedIssuanceTokens * LTV * floorPrice / (1e18 * 10000) >= remainingLoanAmount; collateral withdrawn == requestedLoanAmount; fee sent to treasury. **Access level** External, nonReentrant, permissioned, validAmount
CreditFacility_v1	borrowFor(address receiver_, uint requestedLoanAmount_)	**What this function can do** Same as borrow() but loan is owned by receiver_ and collateral is sent to receiver_; issuance tokens are pulled from caller. **What this function cannot/should not do** Allow receiver_ to be address(0); allow reentrancy. **Main invariant(s)** Same as borrow(); loan.borrower == receiver_. **Access level** External, nonReentrant, permissioned, validAmount, validAddress

Contract Name	Function	Category
CreditFacility_v1	buyAndBorrow(uint amount_, uint loops_, bool consolidate_, uint minAmountOut_)	**What this function can do** Transfer collateral from caller; in a loop: buy issuance tokens from Floor bonding curve, then borrow against them; optionally consolidate all resulting loans; return remaining collateral to caller. **What this function cannot/should not do** Execute more loops than maxLoops or zero loops; allow reentrancy; mint below minAmountOut_ (slippage protection). **Main invariant(s)** Total issuance tokens minted >= minAmountOut_; each loop creates a valid loan; remaining collateral returned to caller. **Access level** External, nonReentrant, permissioned, validAmount, loopCountIsValid
CreditFacility_v1	buyAndBorrowFor(address receiver_, uint amount_, uint loops_, bool consolidate_, uint minAmountOut_)	**What this function can do** Same as buyAndBorrow() but loans are owned by receiver_; remaining collateral returned to caller (not receiver_). **What this function cannot/should not do** Allow receiver_ to be address(0); bypass loop or amount validations. **Main invariant(s)** Same as buyAndBorrow(); all loans owned by receiver_. **Access level** External, nonReentrant, permissioned, validAddress, validAmount, loopCountIsValid

Contract Name	Function	Category
CreditFacility_v1	repay(uint loanId_, uint repaymentAmount_)	**What this function can do** Accept collateral repayment from caller; deposit collateral back to Floor; unlock proportional (partial) or all (full) issuance tokens; deactivate loan on full repayment. **What this function cannot/should not do** Repay a loan not owned by caller (loanIdIsValid checks borrower == msgSender); repay zero amount; allow reentrancy. **Main invariant(s)** After partial repay: loan.remainingLoanAmount decreases by repaymentAmount; issuance tokens unlocked proportionally. After full repay: loan.isActive = false; all locked tokens returned. **Access level** External, nonReentrant, permissioned, loanIdIsValid, validAmount
CreditFacility_v1	transferLoan(uint loanId_, address newBorrower_)	**What this function can do** Transfer loan ownership from caller to newBorrower_; update loan.borrower; move loanId from caller's to newBorrower_'s loan list. **What this function cannot/should not do** Transfer to self; transfer to address(0); transfer a loan not owned by caller; allow reentrancy. **Main invariant(s)** Loan ownership changes atomically; no tokens move; loan state unchanged except borrower field. **Access level** External, nonReentrant, permissioned, loanIdIsValid, validAddress

Contract Name	Function	Category
CreditFacility_v1	rebalanceLoan(uint loanId_)	**What this function can do** When floor price has increased, calculate additional borrowing power; withdraw additional collateral from Floor; charge fee; send net to borrower; increase remainingLoanAmount. **What this function cannot/should not do** Rebalance inactive loans (returns 0); rebalance when remaining >= releasable (reverts NothingToRebalance); rebalance a loan not owned by caller. **Main invariant(s)** New remainingLoanAmount == releasableCollateralAmount (based on current floor and LTV); additional collateral sent to borrower minus fee. **Access level** External, nonReentrant, permissioned, loanIdIsValid
CreditFacility_v1	consolidateLoans(uint[] calldata loanIds_)	**What this function can do** Merge multiple active loans owned by caller into a single new loan; deactivate old loans; sum up locked tokens and remaining amounts. **What this function cannot/should not do** Consolidate fewer than 2 loans; consolidate loans not owned by caller or inactive loans; allow duplicate loan IDs. **Main invariant(s)** New loan's lockedIssuanceTokens == sum of old; new remainingLoanAmount == sum of old; all old loans deactivated. **Access level** External, nonReentrant, permissioned

Contract Name	Function	Category
CreditFacility_v1	setLoanToValueRatio(uint newLoanToValueRatio_)	**What this function can do** Update LTV ratio; emit warning if lowered (risk of underwater loans). **What this function cannot/should not do** Set LTV to 0 or > 10000 bps. **Main invariant(s)** _loanToValueRatio is within (0, 10000). **Access level** External, permissioned (admin)
CreditFacility_v1	setMaxLoops(uint newMaxLoops_)	**What this function can do** Update the maximum loop count for buyAndBorrow. **What this function cannot/should not do** Set to 0 or > 255. **Main invariant(s)** _maxLoops is within (1, 255). **Access level** External, permissioned (admin)
CreditFacility_v1	setBorrowingFeeRate(uint newFeeRate_)	**What this function can do** Update the borrowing fee rate. **What this function cannot/should not do** Set fee > 5000 bps (50%). **Main invariant(s)** _borrowingFeeRate <= 5000. **Access level** External, permissioned (admin)

Presale_v1

Contract Name	Function	Category
Presale_v1	init(address floor_, address authorizer_, address feeTreasury_, bytes memory configData_)	**What this function can do** Initialize the presale module with credit facility address, commission schedule, end timestamp, global/per-address issuance caps, price breakpoints, initial multiplier, and decay duration. **What this function cannot/should not do** Be called more than once; accept invalid commission bps (> 50%); accept mismatched breakpoint array lengths; accept end timestamp in the past. **Main invariant(s)** Module initialized exactly once; all parameters within valid bounds; state starts as NotOpen. **Access level** External, initializer (once via proxy)
Presale_v1	getPresaleState()	**What this function can do** Return the current presale state; automatically return Closed if end timestamp has passed. **What this function cannot/should not do** Modify state. **Main invariant(s)** Returns Closed when block.timestamp >= _endTimestamp (if set), otherwise returns stored _state. **Access level** Public view, unrestricted

Contract Name	Function	Category
Presale_v1	getCreditFacility()	**What this function can do** Return the credit facility address. **What this function cannot/should not do** Modify state. **Main invariant(s)** Returns stored _creditFacility. **Access level** Public view, unrestricted
Presale_v1	getGlobalIssuanceCap()	**What this function can do** Return the global issuance cap. **What this function cannot/should not do** Modify state. **Main invariant(s)** Returns stored _globalIssuanceCap. **Access level** Public view, unrestricted
Presale_v1	getPerAddressIssuanceCap()	**What this function can do** Return the per-address issuance cap. **What this function cannot/should not do** Modify state. **Main invariant(s)** Returns stored _perAddressIssuanceCap. **Access level** Public view, unrestricted

Contract Name	Function	Category
Presale_v1	getEndTimestamp()	**What this function can do** Return the presale end timestamp. **What this function cannot/should not do** Modify state. **Main invariant(s)** Returns stored _endTimestamp. **Access level** Public view, unrestricted
Presale_v1	getBaseCommissionBps()	**What this function can do** Return the commission bps array. **What this function cannot/should not do** Modify state. **Main invariant(s)** Returns stored _baseCommissionBps. **Access level** Public view, unrestricted
Presale_v1	getPriceBreakpoints()	**What this function can do** Return the price breakpoints 2D array. **What this function cannot/should not do** Modify state. **Main invariant(s)** Returns stored _priceBreakpoints. **Access level** Public view, unrestricted

Contract Name	Function	Category
Presale_v1	Returns stored _priceBreakpoints.	**What this function can do** Return a position by ID. **What this function cannot/should not do** Modify state; return for invalid IDs (reverts). **Main invariant(s)** Reverts for positionId_ == 0 or > _posId. **Access level** Public view, unrestricted
Presale_v1	getPositionCount()	**What this function can do** Return total number of positions created. **What this function cannot/should not do** Modify state. **Main invariant(s)** Returns _posId. **Access level** Public view, unrestricted
Presale_v1	getPositionsByOwner(address owner_)	**What this function can do** Return array of position IDs owned by an address. **What this function cannot/should not do** Modify state. **Main invariant(s)** Returns all position IDs in positionsByAddress[owner]. **Access level** Public view, unrestricted

Contract Name	Function	Category
Presale_v1	getGlobalIssuance()	**What this function can do** Return cumulative global issuance. **What this function cannot/should not do** Modify state; return for invalid IDs (reverts). **Main invariant(s)** Returns _globalIssuance. **Access level** Public view, unrestricted
Presale_v1	getIssuanceBy(address account_)	**What this function can do** Return cumulative issuance for a specific address. **What this function cannot/should not do** Modify state. **Main invariant(s)** Returns issuanceBy[account]. **Access level** Public view, unrestricted
Presale_v1	getPositionState(uint positionId_)	**What this function can do** Return claimable, locked, and claimed token amounts for a position by checking direct tokens and each loan tranche's claimability. **What this function cannot/should not do** Modify state; return for invalid IDs (reverts). **Main invariant(s)** claimableTokens + lockedTokens + claimedTokens == position.totalMinted. **Access level** Public view, unrestricted

Contract Name	Function	Category
Presale_v1	getUserTotalTokens(address user_)	**What this function can do** Aggregate claimable, locked, and claimed across all of a user's positions. **What this function cannot/should not do** Modify state. **Main invariant(s)** Sums are consistent with individual position states. **Access level** Public view, unrestricted
Presale_v1	buyPresale(uint deposit_, uint minAmountOut_)	**What this function can do** a direct presale purchase: take deposit from caller, deduct commission fee (with decaying multiplier), send fee to treasury, buy issuance tokens from Floor bonding curve, create a direct position, update global/per-address issuance. **What this function cannot/should not do** Execute when presale is NotOpen or Closed or past end timestamp; execute when caller not whitelisted during Whitelist phase; exceed global or per-address issuance caps; mint below minAmountOut_ (slippage); allow fee to consume entire deposit. **Main invariant(s)** Tokens minted >= minAmountOut_; _globalIssuance + tokensMinted <= _globalIssuanceCap (if cap > 0); _issuanceBy[buyer] + tokensMinted <= _perAddressIssuanceCap (if cap > 0). **Access level** External, nonReentrant, checkPresaleState, validAmount

Contract Name	Function	Category
Presale_v1	buyPresaleWithLoops(uint deposit_, uint loopCount_, uint minAmountOut_)	**What this function can do** Execute a looped presale purchase: take deposit, deduct commission, execute buyAndBorrow via CreditFacility with specified loop count, buy remaining collateral as direct tokens, create a looped position with loan tranches. **What this function cannot/should not do** Execute with loopCount_ == 0 or >= baseCommissionBps.length; exceed issuance caps; mint below minAmountOut; execute when presale is not active. **Main invariant(s)** Total minted (locked in loans + direct) >= minAmountOut_; issuance caps enforced; position correctly tracks all loan IDs and direct tokens. **Access level** External, nonReentrant, checkPresaleState, validAmount

Contract Name	Function	Category
Presale_v1	setPresaleState(PresaleState state_)	**What this function can do** Transition presale state through NotOpen -> Whitelist -> Public -> Closed (and reopen from Closed to Whitelist/Public); start fee decay when going Public. **What this function cannot/should not do** Invalid transitions (e.g., Public -> Whitelist forward); skip state validation. **Main invariant(s)** State transitions follow valid paths; decay starts at most once on first Public transition. **Access level** External, permissioned (admin)
Presale_v1	setCreditFacility(address creditFacility_)	**What this function can do** Update the credit facility address. **What this function cannot/should not do** Change after presale has started (whenNotStarted guard); set to address(0). **Main invariant(s)** Only settable when state == NotOpen. **Access level** External, permissioned (admin), whenNotStarted
Presale_v1	setCaps(uint globalCap_, uint perAddressCap_)	**What this function can do** Update global and per-address issuance caps. 0 disables respective cap. **What this function cannot/should not do** N/A (no validation beyond permissioned). **Main invariant(s)** Caps are stored as provided. **Access level** External, permissioned (admin)

Contract Name	Function	Category
Presale_v1	setEndTimestamp (uint64 endTimestamp_)	**What this function can do** Set or update the presale end timestamp. 0 disables time-based ending. **What this function cannot/should not do** Set a timestamp in the past (reverts if endTimestamp_ > 0 && endTimestamp_ <= block.timestamp). **Main invariant(s)** If set, _endTimestamp > block.timestamp. **Access level** External, permissioned (admin)
Presale_v1	setBaseCommissionBpsAndPriceBreakpoints(uint16[] calldata baseCommissionBps_, uint[][] calldata priceBreakpoints_)	**What this function can do** Update commission schedule and price breakpoints. **What this function cannot/should not do** Modify after presale has started (reverts if state != NotOpen); accept bps > 50%; accept mismatched array lengths; accept non-ascending breakpoints. **Main invariant(s)** Only modifiable before presale starts; all bps <= 5000; breakpoints strictly ascending; priceBreakpoints.length == baseCommissionBps.length - 1. **Access level** External, permissioned (admin)

Contract Name	Function	Category
Presale_v1	setMerkleRoot(bytes32 merkleRoot_)	**What this function can do** Set the Merkle root for whitelist verification. **What this function cannot/should not do** Modify after presale has started (whenNotStarted guard). **Main invariant(s)** Only settable when state == NotOpen. **Access level** External, permissioned (admin), whenNotStarted
Presale_v1	setInitialMultiplier(uint32 multiplier_)	**What this function can do** Update the initial fee multiplier for decay calculation. **What this function cannot/should not do** Bypass permissioned check. **Main invariant(s)** Multiplier stored as provided. **Access level** External, permissioned (admin)
Presale_v1	setDecayDuration(uint64 duration_)	**What this function can do** Update the decay duration for fee multiplier. **What this function cannot/should not do** Bypass permissioned check. **Main invariant(s)** Duration stored as provided. **Access level** External, permissioned (admin)

Contract Name	Function	Category
Presale_v1	setStartTime(uint64 startTime_)	**What this function can do** Update the decay start time. **What this function cannot/should not do** Bypass permissioned check. **Main invariant(s)** Start time stored as provided. **Access level** External, permissioned (admin)

StakingManager_v1

Contract Name	Function	Category
StakingManager_v1	init(address floor_, address authorizer_, address feeTreasury_, bytes memory configData_)	**What this function can do** Initialize the module with performance fee bps; fetch collateral/issuance tokens from Floor. **What this function cannot/should not do** Be called more than once; accept performance fee > 10000 bps. **Main invariant(s)** Module initialized exactly once; _performanceFeeBps <= 10000. **Access level** External, initializer (once via proxy)
StakingManager_v1	getPosition(address user_, address strategy_)	**What this function can do** Return a user's position in a strategy. **What this function cannot/should not do** Modify state. **Main invariant(s)** Returns stored position data. **Access level** Public view, unrestricted
StakingManager_v1	getPositionValue(address user_, address strategy_)	**What this function can do** Return current value of a position by calling strategy's convertToAssets on user's share balance **What this function cannot/should not do** Modify state. **Main invariant(s)** Value reflects strategy's current share-to-asset conversion. **Access level** Public view, unrestricted

Contract Name	Function	Category
StakingManager_v1	getAvailableYield(address user_, address strategy_)	**What this function can do** Return available yield (currentValue - deployedCollateral) for a position. **What this function cannot/should not do** yield = max(0, currentValue - deployedCollateral). **Main invariant(s)** yield = max(0, currentValue - deployedCollateral). **Access level** Public view, unrestricted
StakingManager_v1	isStrategyApproved(address strategy_)	**What this function can do** Return whether a strategy is approved. **What this function cannot/should not do** yield = max(0, currentValue - deployedCollateral). **Main invariant(s)** Returns approvedStrategies[strategy]. **Access level** Public view, unrestricted
StakingManager_v1	getPerformanceFeeBps()	**What this function can do** Return the performance fee in bps. **What this function cannot/should not do** yield = max(0, currentValue - deployedCollateral). **Main invariant(s)** Returns stored _performanceFeeBps. **Access level** Public view, unrestricted

Contract Name	Function	Category
StakingManager_v1	stake(address strategy_, uint issuanceTokenAmount_)	**What this function can do** Accept issuance tokens from caller; calculate collateral at current floor price; withdraw collateral from Floor; deposit collateral into ERC4626 strategy (shares credited to user); update user position. If existing position, auto-rebalance first. **What this function cannot/should not do** Stake to unapproved strategy; stake zero amount; allow reentrancy; bypass access control. **Main invariant(s)** deployedCollateral == lockedIssuanceTokens * lastFloorPrice / 1e18 (after rebalance); strategy shares credited to user correspond to deposited collateral **Access level** External, permissioned, nonReentrant, strategyIsApproved, validAmount
StakingManager_v1	harvestYield(address strategy_, address receiver_)	**What this function can do** Calculate yield (currentValue - deployedCollateral); withdraw yield from strategy; take performance fee; send fee to treasury; send net yield to receiver. **What this function cannot/should not do** Harvest from unapproved strategy; harvest without a position; harvest zero yield (reverts); send to address(0); allow reentrancy. **Main invariant(s)** fee = actualReceived * performanceFeeBps / 10000; netYield = actualReceived - fee; position's deployedCollateral unchanged. **Access level** External, permissioned, nonReentrant, strategyIsApproved, hasPosition, validAddress

Contract Name	Function	Category
StakingManager_v1	withdrawFunds(address strategy_, uint collateralAmount_, address receiver_)	**What this function can do** Auto-harvest pending yield first; withdraw specified collateral from strategy; deposit collateral back to Floor; calculate proportional issuance tokens to return; transfer issuance tokens to receiver; update position. **What this function cannot/should not do** Withdraw more than position value; withdraw from unapproved strategy; withdraw without position; withdraw zero; send to address(0); allow reentrancy; accept incomplete withdrawal from strategy. **Main invariant(s)** issuanceTokensToReturn = lockedIssuanceTokens * collateralAmount_ / deployedCollateral; position.deployedCollateral and lockedIssuanceTokens decrease proportionally. **Access level** External, permissioned, nonReentrant, strategyIsApproved, hasPosition, validAmount, validAddress

Contract Name	Function	Category
StakingManager_v1	rebalance(address strategy_)	**What this function can do** When floor price has increased, withdraw additional collateral from Floor and deploy it to strategy; update position's deployedCollateral and lastFloorPrice. **What this function cannot/should not do** Rebalance when floor price hasn't increased (reverts NothingToRebalance); rebalance without a position; rebalance to unapproved strategy. **Main invariant(s)** After rebalance: deployedCollateral == lockedIssuanceTokens * currentFloorPrice / 1e18; lastFloorPrice updated to current. **Access level** External, permissioned, nonReentrant, strategyIsApproved, hasPosition
StakingManager_v1	addStrategy(address strategy_)	**What this function can do** Approve a new strategy after verifying: implements IStrategyBase_v1 (ERC165), asset matches collateral token, shares are non-transferable. **What this function cannot/should not do** Add address(0); add already-approved strategy; add strategy with wrong asset; add strategy with transferable shares. **Main invariant(s)** Only strategies meeting all three checks get approved. **Access level** External, permissioned (admin), validAddress

Contract Name	Function	Category
StakingManager_v1	removeStrategy(address strategy_)	**What this function can do** Remove approval for a strategy. **What this function cannot/should not do** Remove unapproved strategy; remove strategy that still has deposited value (totalAssets > 0); remove address(0). **Main invariant(s)** Strategy can only be removed when empty. **Access level** External, permissioned (admin), validAddress
StakingManager_v1	setPerformanceFeeBps(uint feeBps_)	**What this function can do** Update the performance fee percentage. **What this function cannot/should not do** Set to 0 (validAmount); set > 10000 bps. **Main invariant(s)** performanceFeeBps within (0, 10000]. **Access level** External, permissioned (admin), validAmount

Floor_v1 / BC_Discrete_Redeeming_VirtualSupply_v1

Contract Name	Function	Category
BC_DRVSV1	init(address floor_, address authorizer_, address feeTreasury_, bytes memory configData_)	**What this function can do** Initialize the bonding curve with issuance token, collateral token, initial segments array, buy fee, and sell fee. **What this function cannot/should not do** Be called more than once; accept segments that fail _validateSegmentArray checks. **Main invariant(s)** Module initialized exactly once; _segments set and validated; _issuanceToken and _collateralToken non-zero; fees ⩽ 10000 bps. **Access level** External, initializer (once via proxy)
BC_DRVSV1	getSegments()	**What this function can do** Return the full _segments array defining the bonding curve shape. **What this function cannot/should not do** Modify state. **Main invariant(s)** Returns stored position data. **Access level** Public view, unrestricted
BC_DRVSV1	getStaticPriceForBuying()	**What this function can do** Return the current buy price by finding the position for totalSupply + 1 in the segments. **What this function cannot/should not do** Modify state. **Main invariant(s)** Price reflects the next token's cost on the discrete curve. **Access level** Public view, unrestricted

Contract Name	Function	Category
BC_DRVSV1	getStaticPriceForSelling()	**What this function can do** Return the current sell price by finding the position for totalSupply in the segments. **What this function cannot/should not do** Modify state. **Main invariant(s)** Price reflects the current token's redemption value on the discrete curve. **Access level** Public view, unrestricted
BC_DRVSV1	buyFor(address receiver_, uint depositAmount_, uint minAmountOut_)	**What this function can do** Accept collateral from caller; mint issuance tokens to receiver via bonding curve math; deduct buy fee to treasury; update _virtualCollateralSupply via balance-diff. **What this function cannot/should not do** Mint when buying is disabled; mint below minAmountOut_; allow reentrancy. **Main invariant(s)** _virtualCollateralSupply increases by exact balance change (SYNC with real balance); _segments unchanged (by design — supply position computed dynamically); _calculateReserveForSupply(_segments, newTotalSupply) ⩽ _virtualCollateralSupply. **Access level** Public, buyingIsEnabled, validReceiver, permissioned, nonReentrant

Contract Name	Function	Category
BC_DRVSV1	buy(uint depositAmount_, uint minAmountOut_)	**What this function can do** Same as buyFor but receiver is _msgSender(). **What this function cannot/should not do** Same constraints as buyFor. **Main invariant(s)** Same as buyFor. **Access level** Public, buyingIsEnabled, permissioned, nonReentrant
BC_DRVSV1	sellTo(address receiver_, uint depositAmount_, uint minAmountOut_)	**What this function can do** Accept issuance tokens from caller; burn them; transfer collateral to receiver via bonding curve math; deduct sell fee to treasury; update _virtualCollateralSupply via balance-diff. **What this function cannot/should not do** Sell when selling is disabled; return below minAmountOut_; allow reentrancy. **Main invariant(s)** _virtualCollateralSupply decreases by exact balance change (SYNC); _segments unchanged in BC_DRVSV1 (CLUSTER_GAP — no floor adjustment); safeTransfer reverts if real balance < computed payout (risk when withdrawCollateralTo has drained real balance while virtual stays inflated). **Access level** Public, sellingIsEnabled, validReceiver, permissioned, nonReentrant

Contract Name	Function	Category
BC_DRVSV1	sell(uint depositAmount_, uint minAmountOut_)	**What this function can do** Same as sellTo but receiver is _msgSender(). **What this function cannot/should not do** Same constraints as sellTo. **Main invariant(s)** Same as sellTo **Access level** Public, sellingIsEnabled, permissioned, nonReentrant
BC_DRVSV1	setVirtualCollateralSupply(uint virtualSupply_)	**What this function can do** Admin override of _virtualCollateralSupply to any value. **What this function cannot/should not do** Bypass permissioned check. **Main invariant(s)** sets virtual supply without checking real balance. If set higher than balanceOf(address(this)), creates phantom reserve — the curve prices sells as if more collateral exists than is actually held, eventually causing sell reverts. If set lower, curve under-reports reserves (conservative, safe direction). **Access level** External, permissioned (admin)

Contract Name	Function	Category
BC_DRVSV1	reconfigureSegments(PackedSegment[] calldata newSegments_, uint suppliedCollateral_, bool selfSupplied_)	**What this function can do** Replace the entire _segments array; optionally accept new collateral (external transfer or self-supplied from existing balance surplus). Enforces invariant: _calculateReserveForSupply(newSegments, totalSupply) ⩽ _virtualCollateralSupply. **What this function cannot/should not do** Accept segments where required reserve exceeds virtual supply (reverts); accept self-supplied collateral exceeding actual balance surplus (reverts). **Main invariant(s)** _calculateReserveForSupply(newSegments, totalSupply) ⩽ _virtualCollateralSupply after the call. RISK: if _virtualCollateralSupply is stale-high (post-withdrawCollateralTo), this check uses inflated virtual supply → can accept segments requiring more reserve than actual balance → all subsequent sells revert (DEEP_PROPAGATION DP-1). **Access level** External, permissioned (admin)

Contract Name	Function	Category
BC_DRVSV1	withdrawCollateralTo(uint amount_)	**What this function can do** Transfer real collateral out to caller (CreditFacility or StakingManager). INTENTIONALLY does NOT decrease _virtualCollateralSupply (per natspec: "for interactions that keep collateral in the Floor System for later return"). **What this function cannot/should not do** Withdraw zero amount (reverts). **Main invariant(s)** After call: _virtualCollateralSupply > balanceOf(address(this)) is possible and expected. **Access level** External, permissioned (CreditFacility, StakingManager)
BC_DRVSV1	depositCollateralFrom(uint amount_)	**What this function can do** Accept collateral from caller (CreditFacility repaying loan, StakingManager returning collateral). INTENTIONALLY does NOT increase _virtualCollateralSupply (per natspec). **What this function cannot/should not do** Deposit zero amount (reverts). **Main invariant(s)** After call: balanceOf(address(this)) > _virtualCollateralSupply (surplus). **Access level** External, permissioned (CreditFacility, StakingManager)

Contract Name	Function	Category
Floor_v1	getFloorSection()	**What this function can do** Return _segments[0] — the floor segment. **What this function cannot/should not do** Modify state. **Main invariant(s)** Floor segment has 1 step and 0 price increase. **Access level** Public view, unrestricted
Floor_v1	getPremiumSections()	**What this function can do** Return all segments except the floor (index 1 onwards). **What this function cannot/should not do** Modify state. **Main invariant(s)** Returns _segments[1..n]. **Access level** Public view, unrestricted
Floor_v1	getFloorPrice()	**What this function can do** Return _segments[0]._initialPrice() — the floor price. **What this function cannot/should not do** Modify state. **Main invariant(s)** Returns the floor segment's initial price. Not affected by _segments[0].supplyPerStep staleness — reads _initialPrice only. **Access level** Public view, unrestricted

Contract Name	Function	Category
Floor_v1	getFloorMetrics()	**What this function can do** Return floor price, floor supply (capacity), current total supply, and next premium price. **What this function cannot/should not do** Modify state. **Main invariant(s)** floorSupply_ is the configured floor capacity (_segments[0].supplyPerStep), not necessarily equal to currentSupply_. These are semantically different (capacity vs actual). **Access level** Public view, unrestricted
Floor_v1	sellTo(address receiver_, uint depositAmount_, uint minAmountOut_)	**What this function can do** Same as BC_DRVSV1 sellTo but additionally: after the sell, if totalSupply < _segments[0]._supplyPerStep(), calls _adjustFloorToSupply() to shrink floor segment capacity to match actual supply. **What this function cannot/should not do** Same constraints as BC_DRVSV1 sellTo. **Main invariant(s)** _virtualCollateralSupply always updated via balance-diff (SYNC). _segments[0].supplyPerStep CONDITIONALLY updated — only when totalSupply < supplyPerStep (ASYMMETRIC_BRANCH). When sell stays within floor range (totalSupply ⩾ supplyPerStep), supplyPerStep is NOT updated. This is safe: all downstream consumers read _initialPrice (unaffected) or cap to actualIssuanceSupply **Access level** Public, sellingIsEnabled, validReceiver, permissioned, nonReentrant

Contract Name	Function	Category
Floor_v1	sell(uint depositAmount_, uint minAmountOut_)	**What this function can do** Same as Floor_v1 sellTo but receiver is _msgSender(). Uses getFloorSection()._supplyPerStep() (equivalent to _segments[0]._supplyPerStep()). **What this function cannot/should not do** Modify state. **Main invariant(s)** Same as Floor_v1 sellTo. **Access level** Public view, unrestricted
Floor_v1	setVirtualCollateralSupply(uint virtualSupply_)	**What this function can do** Same as BC_DRVSV1 setVirtualCollateralSupply — admin override of virtual supply. **What this function cannot/should not do** Same constraints. **Main invariant(s)** Same SYNC_GAP as BC_DRVSV1 version. **Access level** External, permissioned (admin)

Contract Name	Function	Category
Floor_v1	raiseFloor(uint collateralAmount_)	**What this function can do** Increase floor price by injecting collateral and absorbing premium steps. Iteratively absorbs premium steps while affordable; distributes excess collateral over floor supply; calculates exact reserve needed; transfers only the exact amount (dust remains with caller). Uses _reconfigureSegments to apply changes. **What this function cannot/should not do** Raise with zero amount (validAmount); raise when < 2 segments exist; raise when collateral is too small to cause even 1 wei price increase; raise when new reserve ⩽ current virtual supply (no actual increase). **Main invariant(s)** After raise: new floor price > old floor price; _calculateReserveForSupply(newSegments, totalSupply) ⩽ _virtualCollateralSupply; floor segment has 1 step and 0 price increase; floor supply capped to actualIssuanceSupply if supply constraint is hit (_segmentAbsorption capped mode). **Access level** External, permissioned (admin), validAmount

Contract Name	Function	Category
IssuanceBase_v2	enableBuy()	**What this function can do** Set _buyIsOpen = true, enabling all buy operations. **What this function cannot/should not do** Bypass permissioned check. **Main invariant(s)** _buyIsOpen is true after call. **Access level** External, permissioned (admin)
IssuanceBase_v2	disableBuy()	**What this function can do** Set _buyIsOpen = false, disabling all buy operations. **What this function cannot/should not do** Bypass permissioned check. **Main invariant(s)** _buyIsOpen is false after call. **Access level** External, permissioned (admin)
IssuanceBase_v2	setBuyFee(uint fee_)	**What this function can do** Update the buy fee rate. **What this function cannot/should not do** Set fee > 10000 bps; bypass permissioned check. **Main invariant(s)** _buyFee ⩽ 10000. **Access level** External, permissioned (admin)

Contract Name	Function	Category
RedeemingIssuanceBase_v2	enableSell()	**What this function can do** Set _sellIsOpen = true, enabling all sell operations. **What this function cannot/should not do** Bypass permissioned check. **Main invariant(s)** _buyIsOpen is true after call. **Access level** sellIsOpen is true after call.
RedeemingIssuanceBase_v2	disableSell()	**What this function can do** et _sellIsOpen = false, disabling all sell operations. **What this function cannot/should not do** Bypass permissioned check. **Main invariant(s)** _sellIsOpen is false after call. **Access level** External, permissioned (admin)
RedeemingIssuanceBase_v2	setSellFee(uint fee_)	**What this function can do** Update the sell fee rate. **What this function cannot/should not do** Set fee > 10000 bps; bypass permissioned check. **Main invariant(s)** sellFee ⩽ 10000. **Access level** External, permissioned (admin)

Contract Name	Function	Category
VirtualCollateralSupplyBase_v1	getVirtualCollateralSupply()	**What this function can do** Return _virtualCollateralSupply. **What this function cannot/should not do** Modify state. **Main invariant(s)** Returns the tracked virtual supply. Note: after withdrawCollateralTo this value may exceed actual balance (inflated). After depositCollateralFrom this may be below actual balance (conservative). External consumers should be aware of possible divergence. **Access level** Public view, unrestricted

3. Asset Flow Mapping (Critical Contracts)

This section identifies where money actually lives and how it moves.

3.1 Asset-Holding Contracts

List only contracts that custody value.

Contract	Asset Type	Custodied Assets
CreditFacility_v1	ERC20 (Issuance Token)	Locked issuance tokens from borrowers as loan collateral
CreditFacility_v1	ERC20 (Collateral Token)	Transient: collateral passes through during borrow/repay (withdrawn from Floor, sent to borrower or received from borrower)
Presale_v1	ERC20 (Issuance Token)	Direct purchase tokens held until presale ends and user claims
Presale_v1	ERC20 (Collateral/ Reserve Token)	Transient: deposits pass through to Floor bonding curve or CreditFacility
StakingManager_v1	ERC20 (Issuance Token)	Locked issuance tokens from stakers
StakingManager_v1	ERC20 (Collateral Token)	Transient: collateral passes through during stake/withdraw/harvest
External Strategies (ERC4626)	ERC20 (Collateral Token)	Deployed collateral from StakingManager stakers
Floor_v1 / BC_DRVSV1	ERC20 (Collateral Token)	Bonding curve reserve — collateral backing all issued tokens. _virtualCollateralSupply tracks the curve's view; balanceOf(address(this)) is the actual amount. These can diverge by design (SG-1, SG-2).
Floor_v1 / BC_DRVSV1	ERC20 (Issuance Token)	Not held — issuance tokens are minted to buyers and burned from sellers. The contract is registered as a minter on the ERC20Issuance_v1 token.

3.2 Asset Entry & Exit Functions

This table maps money movement, which is where most exploits happen.

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
CreditFacility_v1	borrow / borrowFor	Issuance tokens (locked)	Collateral tokens (from Floor)	Permissioned user	Floor price manipulation could over-borrow; fee calculation rounding
CreditFacility_v1	buyAndBorrow / buyAndBorrowFor	Collateral tokens (from caller)	Remaining collateral (to caller)	Permissioned user	Multi-loop bonding curve interaction; slippage risk; flash loan concerns
CreditFacility_v1	repay	Collateral tokens (from caller)	Issuance tokens (unlocked to caller)	Loan owner	Proportional unlock rounding on partial repay
CreditFacility_v1	rebalanceLoan	–	Collateral tokens (from Floor, to borrower)	Loan owner	Floor price / LTV change interaction; fee on released collateral
CreditFacility_v1	_executeTokenTransfer	–	Collateral to borrower + fee to Treasury	Internal	Fee to treasury via fetchFunds; net to receiver
Presale_v1	buyPresale	Collateral tokens (deposit)	— (issuance tokens held in contract)	Public/ Whitelisted	Fee calculation with decaying multiplier; bonding curve slippage

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
Presale_v1	buyPresaleWithLoops	Collateral tokens (deposit)	— (issuance tokens in loans + held)	Public/ Whitelisted	Complex: fee → CreditFacility buyAndBorrow → remainder buy; multi-contract interaction
Presale_v1	claimAll	–	Issuance tokens (direct) + Loan transfers	Position owner	Tranche unlock timing; loan transfer to user
Presale_v1	_sendFeesToTreasury	–	Collateral fee to Treasury	Internal	Fee correctly routed
StakingManager_v1	stake	Issuance tokens (from caller)	— (collateral deployed to strategy)	Permissioned user	Floor price used for collateral calculation; auto-rebalance on existing position
StakingManager_v1	harvestYield	–	Collateral tokens (yield to receiver + fee to Treasury)	Position owner	Strategy may return less than expected (rounding); performance fee deducted
StakingManager_v1	withdrawFunds	–	Issuance tokens (to receiver) + Collateral (back to Floor)	Position owner	Proportional issuance token calculation; incomplete withdrawal check; auto-harvest first

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
StakingManager_v1	rebalance	–	– (additional collateral from Floor to strategy)	Position owner	Floor price increase deploys more collateral; no user token movement
StakingManager_v1	_sendFeesToTreasury	–	Performance fee to Treasury	Internal	Fee correctly routed
Floor_v1 / BC_DRVSV1	buyFor / buy	Collateral tokens (from buyer)	Issuance tokens (minted to buyer)	Permissioned user	Bonding curve price manipulation; _virtualCollateralSupply updated via balance-diff (SYNC) but _segments unchanged (CLUSTER_GAP — by design, supply enters premium zone)
Floor_v1 / BC_DRVSV1	sellTo / sell	Issuance tokens (burned from seller)	Collateral tokens (to seller)	Permissioned user	_virtualCollateralSupply updated via balance-diff _segments[0].supplyPerStep CONDITIONALLY updated only if totalSupply < floor capacity ; if collateral was withdrawn via withdrawCollateralTo, safeTransfer may revert (insufficient real balance)

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
Floor_v1 / BC_DRVSV1	withdrawCollateralTo	–	Collateral tokens (to caller module)	Permissioned (CreditFacility, StakingManager)	: real balance decreases but _virtualCollateralSupply is NOT decreased (by design). Creates window where virtual > actual. If _reconfigureSegments is called during this window, invariant check uses inflated virtual supply → can accept segments requiring more reserve than actual balance → all subsequent sells revert
Floor_v1 / BC_DRVSV1	depositCollateralFrom	Collateral tokens (from caller module)	–	Permissioned (CreditFacility, StakingManager)	SYNC_GAP (safe direction): real balance increases but _virtualCollateralSupply unchanged. Surplus consumed by reconfigureSegments (selfSupplied=true)
Floor_v1 / BC_DRVSV1	reconfigureSegments	Collateral tokens (if suppliedCollateral > 0 and ! selfSupplied)	–	Permissioned (admin)	Invariant check: newCalculatedReserve <= _virtualCollateralSupply. If virtual supply is stale-high (post-withdrawCollateralTo), check may pass when it should fail

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
Floor_v1 / BC_DRVSV1	setVirtualCollateralSupply	–	–	Permissioned (admin)	admin can set virtual supply to any value independent of real balance. If set higher than actual → phantom reserve → sells eventually revert
Floor_v1	raiseFloor	Collateral tokens (from caller)	–	Permissioned (admin)	SYNC_GAP (safe direction): real balance increases but _virtualCollateralSupply unchanged. Surplus consumed by reconfigureSegments (selfSupplied=true)
Floor_v1 / BC_DRVSV1	reconfigureSegments	Collateral tokens (if suppliedCollateral > 0 and !selfSupplied)	–	Permissioned (admin)	Absorbs premium steps into floor; exact collateral calculated via _calculateReserveForSupply; uses _reconfigureSegments with suppliedCollateral
IssuanceBase_v2	enableBuy / disableBuy	–	–	Permissioned (admin)	Gates all buy operations via _buyIsOpen flag
IssuanceBase_v2	enableSell / disableSell	–	–	Permissioned (admin)	Gates all sell operations via _sellIsOpen flag
IssuanceBase_v2	setBuyFee	–	–	Permissioned (admin)	Fee routed to treasury on buy; must be ⩽ 10000 bps

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
RedeemingIssuanceBase_v2	setSellFee	–	–	Permissioned (admin)	Fee routed to treasury on sell; must be ⩽ 10000 bps

Closing Summary

In this report, we have considered the security of Floor Finance. We performed our audit according to the procedure described above.

Issues of critical, medium and Informational Severity were found during Audit. The Floors Finance Team acknowledged a few issues and resolved the others.

Disclaimer

At QuillAudits, we have spent years helping projects strengthen their smart contract security. However, security is not a one-time event—threats evolve, and so do attack vectors. Our audit provides a security assessment based on the best industry practices at the time of review, identifying known vulnerabilities in the received smart contract source code.

This report does not serve as a security guarantee, investment advice, or an endorsement of any platform. It reflects our findings based on the provided code at the time of analysis and may no longer be relevant after any modifications. The presence of an audit does not imply that the contract is free of vulnerabilities or fully secure.

While we have conducted a thorough review, security is an ongoing process. We strongly recommend multiple independent audits, continuous monitoring, and a public bug bounty program to enhance resilience against emerging threats.

Stay proactive. Stay secure.

About QuillAudits

QuillAudits is a leading name in Web3 security, offering top-notch solutions to safeguard projects across DeFi, GameFi, NFT gaming, and all blockchain layers. With eight years of expertise, we've secured over 1500 projects globally, averting over $3 billion in losses. Our specialists rigorously audit smart contracts and ensure DApp safety on major platforms like Ethereum, BSC, Arbitrum, Algorand, Tron, Polygon, Polkadot, Fantom, NEAR, Solana, and others, guaranteeing your project's security with cutting-edge practices.

8+ Years of Expertise	**1M+** Lines of Code Audited
50+ Chains Supported	**1500+** Projects Secured

Follow Our Journey

AUDIT REPORT

May 2026

For

Floors

QuillAudits

Canada, India, Singapore, UAE, UK

www.quillaudits.com audits@quillaudits.com